

Cronogramas em Desenvolvimento de Software

Apresentação

O gerenciamento de cronogramas é um dos pilares da gestão de projetos de software, pois permite transformar objetivos estratégicos em atividades executáveis, com prazos, responsáveis, dependências e mecanismos de controle. Diferentemente de projetos tradicionais de engenharia, projetos de software apresentam elevada incerteza, volatilidade de requisitos, dependência de conhecimento especializado e riscos técnicos significativos. Por isso, a construção de cronogramas exige abordagem técnica, adaptativa e orientada a métricas.

Objetivos de Aprendizagem

Ao final desta aula, espera-se que o estudante seja capaz de:

- Compreender os fundamentos do gerenciamento de cronogramas em projetos de software;
- Estruturar um cronograma a partir da decomposição do escopo do projeto;
- Aplicar técnicas de estimativa de esforço e duração;
- Identificar dependências, marcos e caminho crítico;
- Comparar abordagens preditivas e ágeis no planejamento temporal;
- Monitorar e controlar cronogramas com indicadores de desempenho.

1. Conceitos Fundamentais

1.1 O que é um cronograma?

Um cronograma é a representação temporal do planejamento de um projeto, contendo a sequência lógica das atividades, suas durações estimadas, interdependências, marcos de controle e prazos de entrega.

No contexto de desenvolvimento de software, o cronograma funciona como mecanismo de coordenação entre análise, arquitetura, desenvolvimento, testes, implantação e suporte.

1.2 Finalidades do cronograma

Um cronograma bem elaborado permite:

- Planejamento realista das entregas;
- Alocação eficiente de recursos humanos e tecnológicos;
- Identificação antecipada de gargalos;
- Controle de atrasos e desvios;
- Comunicação entre stakeholders;
- Apoio à tomada de decisão gerencial.

2. O Papel do Cronograma no Ciclo de Vida do Software

O cronograma acompanha todas as fases do ciclo de vida do software:

Fase	Objetivo	Entregáveis
Levantamento de requisitos	Identificar necessidades do cliente	Documento de requisitos / backlog
Análise	Modelagem funcional e técnica	Diagramas e especificações
Projeto	Definir arquitetura e design	Arquitetura do sistema
Implementação	Construção do software	Código-fonte
Testes	Garantia da qualidade	Casos e relatórios de teste
Implantação	Disponibilização do sistema	Ambiente produtivo
Manutenção	Correções e melhorias	Releases evolutivas

3. Estrutura Analítica do Projeto (EAP/WBS)

A Estrutura Analítica do Projeto (Work Breakdown Structure) consiste na decomposição hierárquica do escopo em partes menores e gerenciáveis.

Exemplo

Projeto: Sistema Acadêmico

- Planejamento
 - Definição do escopo

- Reunião inicial
- Requisitos
 - Entrevistas com usuários
 - Modelagem de requisitos
- Projeto
 - Arquitetura
 - Banco de dados
 - Protótipos
- Desenvolvimento
 - Módulo de autenticação
 - Módulo acadêmico
 - Módulo financeiro
- Testes
 - Unitários
 - Integração
 - Homologação
- Implantação
 - Publicação
 - Treinamento

Benefícios:

- Melhor precisão nas estimativas;
- Facilidade de monitoramento;
- Melhor atribuição de responsabilidades.

4. Estimativas em Projetos de Software

Estimativa é o processo de prever esforço, duração e custo.

4.1 Técnicas comuns

Analogia

Usa dados históricos de projetos semelhantes.

Vantagem: rapidez. Desvantagem: dependência de histórico confiável.

Julgamento Especializado

Baseado na experiência de profissionais.

Vantagem: flexibilidade. Desvantagem: subjetividade.

Bottom-Up

Estimativa detalhada por atividade.

Vantagem: maior precisão. Desvantagem: maior esforço.

Pontos de Função

Pontos de Função são uma técnica de estimativa usada para medir o **tamanho funcional de um software com base nas funcionalidades entregues ao usuário**, independentemente da linguagem de programação utilizada.

A técnica avalia cinco componentes principais:

- **Entradas Externas (EI):** dados inseridos no sistema (ex.: cadastro de usuário);
- **Saídas Externas (EO):** informações processadas e apresentadas ao usuário (ex.: relatórios);
- **Consultas Externas (EQ):** consultas simples de dados (ex.: busca de registros);
- **Arquivos Lógicos Internos (ILF):** dados mantidos pelo próprio sistema;
- **Arquivos de Interface Externa (EIF):** dados utilizados, mas mantidos por sistemas externos.

Cada componente recebe um peso conforme sua complexidade, permitindo calcular o total de Pontos de Função.

Exemplo:

Um sistema com 100 Pontos de Função e produtividade média de 10 horas por PF exigirá aproximadamente:

$$100 \times 10 = 1000 \text{ horas}$$

Essa estimativa pode ser usada para definir esforço, prazo e cronograma do projeto.

Vantagem: independe da tecnologia utilizada e pode ser aplicada ainda na fase de requisitos.

Story Points

Story Points são uma técnica de estimativa amplamente utilizada em métodos ágeis para medir o esforço relativo necessário para implementar uma funcionalidade.

Diferentemente de estimativas em horas, Story Points consideram fatores como:

- complexidade da tarefa;

- volume de trabalho;
- incerteza;
- esforço técnico.

A estimativa é comparativa, ou seja, uma tarefa recebe mais pontos que outra por ser relativamente mais complexa.

Exemplo:

- Login de usuário → 3 pontos
- Cadastro de cliente → 5 pontos
- Dashboard com gráficos → 13 pontos

Se a equipe possui velocidade média de **20 pontos por sprint**, é possível planejar quantas funcionalidades cabem em cada iteração.

COCOMO

Modelo algorítmico para estimativa de esforço.

Equação simplificada:

$$\text{Esforço} = a \times (\text{KLOC})^b$$

Onde:

- **Esforço** = quantidade estimada de trabalho, normalmente em **pessoa-mês**
- **a** = constante empírica definida pelo tipo do projeto
- **KLOC** = *Kilo Lines of Code* (mil linhas de código)
- **b** = expoente empírico relacionado à complexidade

Significado das variáveis

KLOC

Representa o tamanho estimado do software.

Exemplo:

Se o sistema terá aproximadamente **25.000 linhas de código**:

$$\text{KLOC} = 25$$

a e b

Dependem da categoria do projeto no COCOMO clássico.

Tipo de Projeto a b

Orgânico 2.4 1.05

Semi-acoplado 3.0 1.12

Embutido 3.6 1.20

Tipos de projeto

Orgânico: Projetos menores, equipe experiente, menor complexidade.

Exemplo:

- sistema acadêmico
- sistema comercial
- portal web

Semi-acoplado: Complexidade intermediária.

Exemplo:

- ERP médio
- sistema corporativo com integrações

Embutido Alta complexidade e restrições rígidas.

Exemplo:

- software aeronáutico
- sistemas médicos críticos
- software embarcado automotivo

5. Dependências entre Atividades

Tipos principais:

Finish-to-Start (FS)

Uma atividade só começa quando a anterior termina.

Exemplo: Codificação → Testes

Start-to-Start (SS)

Atividades iniciam juntas.

Exemplo: Desenvolvimento frontend e backend

Finish-to-Finish (FF)

Duas atividades terminam juntas.

Start-to-Finish (SF)

Mais raro em software.

6. Caminho Crítico

O caminho crítico representa a sequência de tarefas que determina a menor duração possível do projeto.

Características:

- Não possui folga;
- Qualquer atraso impacta a entrega final;
- Exige monitoramento contínuo.

Exemplo:

A (2 dias) → B (4 dias) → C (3 dias). Duração total = 9 dias

Se B atrasar 2 dias, o projeto atrasará 2 dias.

Exemplo prático:

Objetivo: O **caminho crítico** é a sequência de atividades que determina a duração mínima do projeto. Qualquer atraso em uma dessas tarefas impactará diretamente a data final de entrega.

Projeto: **Sistema de biblioteca**

Atividade	Duração	Dependência
Levantamento de requisitos	5 dias	—
Modelagem do banco de dados	3 dias	Requisitos
Desenvolvimento backend	8 dias	Banco de dados
Desenvolvimento frontend	6 dias	Requisitos
Integração	4 dias	Backend + Frontend
Testes	5 dias	Integração

Analisando os caminhos:

Caminho 1:

Requisitos → Banco de dados → Backend → Integração → Testes
 $5 + 3 + 8 + 4 + 5 = 25$

Caminho 2:

Requisitos → Frontend → Integração → Testes
 $5 + 6 + 4 + 5 = 20$

Caminho crítico:

Requisitos → Banco de dados → Backend → Integração → Testes

Total:

25 dias

Interpretação: se o desenvolvimento backend atrasar 3 dias, todo o projeto atrasará 3 dias, pois essa atividade faz parte do caminho crítico.

7. Diagramas de Gantt

Ferramenta visual para planejamento temporal.

Permite visualizar:

- duração das tarefas;
- paralelismo;
- marcos;
- progresso;
- dependências.

Aplicações práticas em software:

- acompanhamento executivo;
- comunicação com clientes;
- controle de entregas.

Semana:	1	2	3	4	5	6
Requisitos	■	■				
Análise		■	■			
Projeto		■	■			
Desenvolvimento			■	■	■	
Testes					■	■
Implantação						■

8. Marcos do Projeto

Marcos são eventos importantes sem duração.

Exemplos:

- aprovação dos requisitos;
- finalização da arquitetura;
- conclusão da sprint;
- homologação concluída;
- go-live.

9. Gestão de Recursos

O cronograma depende diretamente da capacidade da equipe.

Fatores críticos:

- disponibilidade dos desenvolvedores;
- especialização técnica;
- férias/licenças;
- dependência de fornecedores;
- multitarefa excessiva.

Problema clássico: Adicionar pessoas em projeto atrasado pode piorar o atraso. (Brooks' Law)

10. Riscos em Cronogramas

Principais riscos:

- escopo mal definido;
- requisitos instáveis;
- subestimação de esforço;
- dívida técnica;
- retrabalho;
- dependências externas;
- indisponibilidade da equipe;
- falhas de comunicação.

Mitigações:

- **Reservas de contingência:**

- No desenvolvimento de um sistema de e-commerce, a integração com o gateway de pagamento foi estimada em **10 dias**, mas o cronograma inclui **3 dias adicionais** para lidar com possíveis falhas de homologação ou ajustes técnicos.

- **Revisões frequentes:**

- Em um projeto de aplicativo mobile, a equipe realiza **reuniões semanais de acompanhamento** para verificar o progresso das funcionalidades, identificar atrasos e ajustar prioridades do cronograma.

- **Métricas de progresso:**

- Em um sistema acadêmico, a equipe monitora a evolução pelo número de funcionalidades concluídas, por exemplo: **12 de 20 módulos implementados (60% concluído)**.

- **Prototipação antecipada:**

- Antes do desenvolvimento completo de um sistema hospitalar, a equipe cria um **protótipo das telas de prontuário eletrônico** para validar requisitos com os usuários e reduzir retrabalho futuro.

11. Cronogramas em Métodos Tradicionais vs Ágeis

Aspecto	Tradicional	Ágil
Planejamento	Detalhado no início	Evolutivo
Mudanças	Mais difíceis	Esperadas
Controle	Baseado no plano	Baseado em inspeção contínua
Horizonte	Longo prazo	Curto prazo
Unidade temporal	Fases	Sprints

12. Indicadores de Controle

Os indicadores de controle permitem acompanhar o andamento do projeto em relação ao planejamento.

- **Planned Value (PV):** valor planejado, ou seja, quanto deveria ter sido concluído até determinada data.
- **Earned Value (EV):** valor realmente entregue.
- **Actual Cost (AC):** custo real gasto.

Schedule Performance Index (SPI):

$$SPI = EV / PV$$

Interpretação:

- **SPI = 1:** conforme o planejado;
- **SPI < 1:** atrasado;
- **SPI > 1:** adiantado.

Exemplo prático em software:

Uma equipe planejou concluir **10 funcionalidades** de um sistema web até o final da sprint.

Até a data prevista:

- planejado (**PV**) = 10 funcionalidades;

- concluído (EV) = 8 funcionalidades.

Cálculo:

$$SPI=8/10=0,8$$

Interpretação: a equipe entregou apenas 80% do que estava planejado, indicando atraso no cronograma.

13. Ferramentas Profissionais

- Microsoft Project
- Jira
- Azure DevOps
- OpenProject
- Trello
- GitHub Projects
- ClickUp
- Monday.com

14. Exercícios

A)

Projeto: Sistema Hospitalar

Equipe:

- 1 gerente
- 2 analistas
- 4 desenvolvedores
- 2 testers

Prazo: 6 meses

Fases:

- requisitos (3 semanas)
- análise/projeto (4 semanas)

- desenvolvimento (10 semanas)
- testes (4 semanas)
- implantação (2 semanas)

Discussão:

Quais riscos podem comprometer esse cronograma?

B)

Elaborar um cronograma completo para um sistema web contendo:

- mínimo de 15 atividades;
- dependências;
- marcos;
- estimativas;
- identificação do caminho crítico;
- representação em Gantt.

Referências

PMI. (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) (7th ed.).

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach.

Sommerville, I. (2016). Software Engineering.

Brooks, F. P. (1995). The Mythical Man-Month.

Cohn, M. (2005). Agile Estimating and Planning.

Kerzner, H. (2022). Project Management: A Systems Approach to Planning, Scheduling, and Controlling.